

The Multi-Day Event Booking System

How entrpy designed, built, and shipped a new booking engine for White Wall Studios — from a single customer request to production in under two days.

Engagement recap · July 12, 2026

1 Executive summary

White Wall Studios needed to sell multi-day events, not just single sessions. Their booking site could only take one appointment at a time, yet real customers were trying to book three- and four-day event windows. Over a tight back-and-forth, entrpy rebuilt the flagship booking flow into a guided multi-day event builder, wired the operational backend behind it (notifications, cleaner scheduling, setup-crew logistics, calendar buffers), verified the whole money path on a staging mirror, and pushed it live.

Client	White Wall Studios — a Greenville, SC photo & event studio (Powdersville flagship + Taylors Mill).
Builder	entrpy, LLC.
The ask	Let customers book a whole multi-day event as one booking — pick a date range, price each day, add event-only extras once, pay together — and make the operations behind it run themselves.
Outcome	A live, guided multi-day event flow on whitewallstudios.co, backed by an automated operations layer. Full-payment path live; the deposit / auto-charge path built and deliberately held dark until final sign-off.
Timeline	First request July 10, 2:23 PM → live July 12, ~1:07 AM. Roughly 35 hours, start to finish.

35 hrs

REQUEST TO LIVE

73

EMAILS, 3 DAYS

51

COMMITTS

1

STAGING DRY-RUN,
THEN PROD

2 Where we started

White Wall's booking site is not a custom backend — it orchestrates three systems: **Acuity** (the source of truth for appointments and availability), **Square** (payment, via custom payment flows), and the studio's own pricing and add-on rules. The architecture is **pay then book**: no appointment exists on the calendar until the customer has actually paid, which keeps the studio's calendar honest and avoids the premature-confirmation problems that Acuity's own scheduler creates.

That system was built for one session at a time. But the client had live demand for **multi-day events** — a customer wanting the studio for October 3rd through 5th, for example, each day with its own start time, priced as a single event with one checkout. The single-session flow simply could not express that. The client's words, at the start:

“Make Event vs Photo/Video the very first choice. If they pick Event, let them build the event as a set of days — each day its own duration and start time — with the event add-ons once, one checkout, and the event deposit.”

That one paragraph hid an entire platform feature. Solving it well meant rethinking the flow, the pricing, the calendar mechanics, and the operations behind every booking.

3 Figuring out the logic

The design was not handed down — it was found, live, through rapid iteration with the client. The shape of the flow changed three times before it was right, and each change came from putting a working version in front of the client on a staging site and reacting to exactly what felt wrong.

The pre-flow gate

Rather than renumber the existing five-step booking machine (risky — it powers every live single-session booking), entrpy added an **additive front gate**: a “What are you booking?” chooser that sets intent and then reveals the existing flow. Photo/Video keeps the proven path byte-for-byte. Event branches into **Single-day** vs **Multi-day**. Nothing about the existing single-session experience changed.

Three builders, until it was right

- **v1 — add a day at a time.** A day-by-day builder on the studio's existing multi-session cart. Functional, but the client wanted the days to feel like one continuous event, not a stack of separate bookings.

- **v2 — time-typed days.** Day one picks a *start time* (not a duration): come in at 9 PM, 8 PM, ... down to 5 AM for a full day, with each option feeding continuously into the next day. Closer, but still built one day at a time.
- **v3 — the range model (the one that shipped).** The client's insight: this should feel like booking a hotel. entrpy rebuilt step two as a single calendar where the customer picks a **start date and an end date**; the event auto-fills the contiguous range — day one at the chosen access time, the middle days as full days, the last day full by default with an optional **early checkout** for a lower price. Airbnb-style, and it locked the dates into a valid contiguous window as a side effect.

This is the part worth underlining: the winning design came out of **collaboration at speed**. Over roughly two days and more than seventy emails — fifty-six on the busiest day alone — every version was deployed to staging, tested by the client as a real customer, and refined within minutes. The final flow is the client's product, arrived at through entrpy's iterations.

4 The front end

The day-one access-time picker

For a multi-day event, day one is not a “duration” — it is an *access time*. The picker relabels the studio's existing session lengths as arrival times (6:00 PM — \$350, ... 5:00 AM — \$980) and frames them as “you get access on day one and keep it continuously until the end of your event.” The same underlying appointment types, reframed for how an event actually runs.

The range calendar and per-day pricing

Pick a start date, then an end date; the span highlights and the event assembles itself. Pricing is per day, and add-ons carry the studio's **progressive multi-day discount** — full price on day one, 15% off day two, 30% off day three and beyond — applied automatically to the eligible gear (chairs, tables, walls, backdrops, PA, TV) while flat items are charged once. A live event summary builds itself as the customer goes, day by day, with the running total and the mandatory cleaning fee shown the moment multi-day is chosen.

Supporting pages

Alongside the flow, entrpy shipped a public **Add-Ons menu** (every extra with photo, price, and description) and a **Floor Plans** page (to-scale layouts for the full space, seated

events, and standing events), both wired into site-wide navigation.

5 The back end — the inner workings

The front end is what the customer sees. The back end is where a multi-day event becomes real appointments, one payment, and a studio that knows what to do. This is the part that had to be exactly right, because it moves money and puts blocks on a real calendar.

One event, N appointments, one charge

A multi-day event is priced as a **cart**. The checkout creates **one Square charge** for the whole event and **one Acuity appointment per day**, writes a single booking record with per-day sessions, and stores the card on file. Each day lands on the calendar at the correct wall-clock time.

The timezone bug we caught before it shipped

A naive datetime (for example `2026-10-03T21:00:00`) is read as UTC on the hosting platform — so a 9 PM event would have quietly booked at 5 PM. `entrypy` fixed this by building every slot as a full ISO-8601 timestamp *with the Eastern offset* (`...T21:00:00-04:00`), and unit-checked both the daylight (`-04:00`) and standard (`-05:00`) cases so October and January both book correctly.

The multi-calendar gotcha

Because the studio's appointment types belong to more than one Acuity calendar, any API call that omits the calendar ID silently routes to the wrong one. Every calendar write in the new flow — each appointment and every buffer block — explicitly passes the calendar ID, closing a class of misroute bug the hard way.

The dead-end we found and fixed

An early integration exposed a real gap: the multi-day builder walked each day through date and time, but the *universal* details every booking needs — contact info, the signed waiver, terms, and the card — were never collected, so the flow dead-ended at a disabled pay button. `entrypy` routed the finished event through those steps once, for the whole event, and the money path completed cleanly.

The operations layer

The client's real requirement was that the operations behind an event should run themselves. `entrypy` built an event-level notification and scheduling layer that fires **once**

per event (never once per day) on every real multi-day booking:

- **Customer recap** — a confirmation with every day of the event and the total.
- **Owner text (via Watson)** — dates, total, customer name, what they are using the space for, headcount, and a line confirming the cleaners were emailed.
- **A second owner text, only when the Setup & Reset Crew is added** — exactly where each item goes (from the intake), the recommended crew start time (two hours before the event), and the back-end reset time keyed to when the event ends.
- **Cleaner email (April)** — keyed to the last day of the event, with a calendar invite. When the crew is added, her window becomes four hours and she is told to arrive 90 minutes after the event ends, so she is never cleaning on top of the crew resetting.

Calendar buffers and the crew

The studio's calendar automatically protects the time an event actually consumes. A **back-end reset buffer** is blocked after the last day — the standard length normally, and **four hours** when the Setup & Reset Crew is added (crew reset plus cleaning). When the crew is added, a **two-hour front-end block** is also placed before day one so they can set up before guests arrive. The **\$150 cleaning fee** is charged once for the whole event (mandatory for any multi-day event, regardless of headcount), not per day.

What we deliberately held back

The client wanted a 60/40 deposit option where the remaining 40% auto-charges 48 hours before the event. entrpy built the full path but **held it dark on the live site**: arming real automatic charges is a go-live safety decision, so until it is finalized, the deposit toggle is hidden on production and the server forces full payment. No customer is ever shown a promise (“auto-charged 48 hours before”) that the system cannot yet keep. In the meantime, an owner reminder two days before any balance is the manual interim. This is the one intentional gate between what is built and what is switched on.

6 How we proved it before shipping

None of this went live on trust. White Wall runs a fully isolated **staging mirror** of the booking system — sandbox payments, a dedicated test calendar, notifications suppressed — so the entire money path can be exercised without touching real customers, real cards, or the real calendar.

entrpy ran a complete **booking dry-run** on staging: a real two-day event with the Setup & Reset Crew added, driven end to end through the live flow to payment. The result

verified every moving part — two appointments created at the right times and prices, the crew's two-hour front block and four-hour back buffer placed correctly, the crew and cleaning fee each charged once, and a total (\$2,230) that matched the charge to the cent. The test data was then cleaned off the calendar. After go-live, the production flow was walked all the way to the pay screen to confirm it renders and behaves correctly — and to confirm the deposit toggle is correctly absent on the live site.

7 Go-live and what is next

On the client's go-ahead — “Let's do it. Push it all live!” — entrpy merged the work and deployed to **whitewallstudios.co**, then verified it on the live site. The multi-day event flow, the per-day pricing, the live summary, the Add-Ons and Floor Plans pages, and the entire operations layer are live and firing on real bookings.

The remaining step is a single, deliberate one: a deposit dry-run and the safety sign-off to arm the 40% auto-charge, at which point the deposit option flips on. Everything else is done.

Live now	Multi-day event flow, day-one access-time picker, range calendar, per-day pricing, live summary, Add-Ons + Floor Plans pages, and the full owner / customer / cleaner / crew operations layer.
Held dark	60/40 deposit + 40% auto-charge (built, gated off on production until final safety sign-off).
Method	Additive design over a proven flow, staging-first verification, one dry-run of the full money path, then production — with the single high-risk money feature deliberately gated.